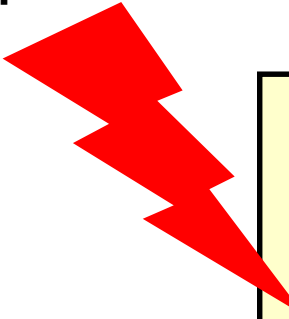


Qu'est ce qu'une Transaction ?

❑ Exemple 1 :

- Vous faites un transfert de 500€ de votre compte épargne au compte courant, cela se traduit dans le code de l'application que vous utilisez par :

Panne quelconque
entre les deux



```
UPDATE CptEpargne SET solde=solde-500  
WHERE ncompte='1423067587' ;
```

```
UPDATE CptCOURANT SET solde=solde+500  
WHERE ncompte='1423067588' ;
```

Qu'est ce qu'une Transaction ?

❑ Exemple 2 :

- Vous réservez une place dans un avion. Cela se traduit dans l'application par un code « similaire à celui-ci » :

Que se passe t-il si une autre personne reserve en même temps que vous?

```
IF (SELECT COUNT(*)  
FROM flyTable  
WHERE siege='libre')>=1)  
THEN  
    UPDATE flytable  
    SET siege=NOMCLI WHERE  
    idsiege IN (SELECT idSiege  
FROM flyTable  
Where siege='libre' LIMIT 1);  
END IF;
```

Qu'est ce qu'une Transaction ?

❑ Solution :

- Faire en sorte que cette suite d'instructions ne fassent qu'une seule: aucune n'est exécutée sans les autres : **c'est ce qu'on appelle une transaction.**
- Une transaction est dite **atomique** : elle ne peut se terminer que par :
 - ✓ un succès (toutes ses opérations sont réalisées) : la transaction est dite validée (**commit**) ou,
 - ✓ un échec (toutes ses opérations sont défaites ou aucune n'est réalisée) : la transaction est annulée (**Rollback**). Il est cependant possible de n'annuler qu'une partie de la transaction en positionnant des points de reprise (**SAVEPOINT**).
- En conséquence, en contexte multi-utilisateurs, les modifications effectuées par une transaction réalisée par un utilisateur ne sont connues des autres utilisateurs que lorsque la transaction a été confirmée par un COMMIT ou abandonnée avec un ROLLBACK.
- **Personne ne voit les modifications sur les tables avant la fin de la transaction**

Propriétés d'une Transaction

□ Propriétés:

En plus de **l'ATOMICITE**, une transaction doit avoir les propriétés suivantes pour une transaction:

- **COHERENCE** : une transaction qui commence avec un ensemble de données cohérent (respectant les contraintes d'intégrité) laisse cet ensemble de données cohérent lorsqu'elle se termine.
- **ISOLATION** : l'exécution concurrente de plusieurs transactions n'affectent pas leur résultat. Tout se passe comme si chaque transaction s'exécute seule ou que toutes les transactions se sont exécutés l'une après l'autre. On dit que les transactions sont **sérialisables**
- **DURABILITE** : le SGBD doit assurer que les effets des transactions réussies soient permanents et ne seront pas affectées par d'éventuelles pannes ou problèmes techniques.

Les 4 propriétés sont réunies sous l'acronyme **ACID**.

Le mode transactionnel sur MARIADB/MYSQL

- ❑ Avec mariaDB/MySQL, le mode transactionnel n'est possible qu'avec le **moteur InnoDB**
- ❑ Deux façons pour passer en mode transactionnel:
 - Positionner la variable du mode transactionnel à 0. Cette variable est par défaut à 1 (chaque commande est une transaction et est validée si elle s'exécute correctement.)
 - ✓ **SET AUTOCOMMIT=0;**
 - On peut également encapsuler juste un ensemble d'instructions en transaction avec
 - ✓ **START TRANSACTION**
 - ✓ Tout en étant dans le mode non transactionnel (c.à.d. sans positionner la variable AUTOCOMMIT à 0)

Transactions

- ❑ Les commandes affectées par le mode transactionnel sont :
 - **INSERT, DELETE et UPDATE**
 - **Pour y rajouter le SELECT, il faut utiliser :**
 - SELECT ... FOR UPDATE (les autres transactions n'ont aucun accès sur les lignes sélectionnées)
 - SELECT ... LOCK IN SHARED MODE (les autres transactions ne peuvent pas modifier les lignes sélectionnées mais peuvent les lire)
 - **Avec autocommit =0;**
 - **Dans ce cas là les lignes concernées par ce SELECT sont verrouillées jusqu'à la fin de la transaction**
 - Les autres commandes
 - ✓ Create, drop, Alter, grant, revoke, etc: sont validées automatiquement à l'exécution comme si on était en mode non transactionnel
 - ✓ **Attention:** Toutes ces commandes valident implicitement les transactions qui les précèdent.

Les anomalies du mode transactionnel

- ❑ Selon le niveau d'isolation instauré par le SGBD, le mode transactionnel **peut** donner lieu aux anomalies suivantes:
 - **Lecture sale (dirty read)**: se produit lorsqu'une transaction est autorisée à lire une ligne qui a été modifiée par une autre transaction en cours d'exécution et qui n'a pas encore été validée.
 - **Lecture non renouvelable (non repeatable read)** : se produit lorsque, au cours d'une transaction, une ligne est récupérée deux fois et que les valeurs de la ligne diffèrent entre les lectures (une modification a eu lieu entre temps).
 - **Lecture fantôme (fantom read)** : se produit lorsque, au cours d'une transaction, deux requêtes identiques sont exécutées et que la collection de lignes renvoyée par la deuxième requête est différente de la première (une transaction concurrente a supprimé ou ajouté des lignes entre temps).

Niveaux d'isolation d'une transaction

- ❑ La sérialisabilité est quelque fois jugé trop contraignante
- ❑ La norme ANSI/ISO SQL défini différents niveaux d'isolation, dont trois plus faibles que le niveau SERIALIZABLE.
- ❑ Ces niveaux sont moins contraignants mais être source d'erreurs ou d'incohérence :
 - READ UNCOMMITTED
 - READ COMMITTED
 - REPETABLE READ
 - SERIALIZABLE
- ❑ SQL : SET TRANSACTION ISOLATION LEVEL niveau

Niveaux d'isolation d'une transaction

Niveau d'Isolation	Lectures sale	Lecture non renouvelable	Lecture Fantôme
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	impossible	possible	possible
REPETABLE READ	impossible	impossible	possible
SERIALIZABLE	impossible	impossible	impossible

- ❑ Sur MARIADB/MySQL le mode par défaut est **REPETABLE READ**